

Day 5: Amazon S3, Identity, and AWS Identity and Access Management (IAM)

Amazon Web Services (AWS) provides a wide range of cloud services to help organizations securely store data and manage user access. Two of the most important services are **Amazon Simple Storage Service (Amazon S3)** and **AWS Identity and Access Management (IAM)**. Together, they enable secure data storage and controlled access to cloud resources.

Amazon Simple Storage Service (Amazon S3) is an object storage service that allows users to store and retrieve any amount of data from anywhere over the Internet. Data is stored as **objects** inside **buckets**, where each object consists of the file, metadata, and a unique identifier. Amazon S3 is designed to provide high durability, scalability, security, and availability, making it suitable for businesses of all sizes.

Amazon S3 is widely used for storing images, videos, documents, backups, application data, software packages, and log files. It also supports static website hosting, disaster recovery, big data analytics, and cloud-native applications. Some key features of Amazon S3 include unlimited storage capacity, automatic scalability, data versioning, lifecycle management, encryption, and data replication across multiple AWS Availability Zones. These features ensure that data remains secure, available, and easily accessible whenever required.

Identity in AWS refers to a digital identity that represents a person, application, or service interacting with AWS resources. Every identity has specific permissions that determine which AWS services and resources it can access. Proper identity management helps organizations protect sensitive information, prevent unauthorized access, and maintain compliance with security policies.

To manage identities securely, AWS provides **Identity and Access Management (IAM)**. IAM is a web service that enables administrators to create and manage users, groups, and roles while controlling access to AWS resources through permission policies. IAM follows the **principle of least privilege**, which means users receive only the permissions necessary to perform their assigned tasks.

The main components of IAM include **Users**, **Groups**, **Roles**, and **Policies**. An **IAM User** represents an individual person or application that requires access to AWS. A **Group** is a collection of users who share the same permissions, making administration easier. An **IAM Role** provides temporary permissions that can be assumed by AWS services or users without requiring long-term credentials. **Policies** are JSON documents that define what actions are allowed or denied on specific AWS resources.

IAM also supports advanced security features such as **Multi-Factor Authentication (MFA)**, password policies, access keys for programmatic access, and detailed activity logging through AWS CloudTrail. These features improve account security and help organizations monitor user activities.

In conclusion, Amazon S3 provides secure, durable, and highly scalable cloud storage for a wide variety of applications, while AWS IAM ensures that only authorized users and services can access cloud resources. Together, Amazon S3 and IAM form the foundation of secure cloud storage and identity management in AWS, helping organizations build reliable, scalable, and secure cloud-based solutions.

Free Cloud Computing Services

Environment overview - events

Roles | IAM | Global

Ask Gemini

nsole.aws.amazon.com/elasticbeanstalk/home?region=us-east-1#/environment/dashboard?environmentId=e-h2hih32u8m

h[Alt+S]

United States (N. Virginia)

Meenakshi M (15519045595)

Meenaks

enments > Meenakshi-nodejs-app-env

Elastic Beanstalk is launching your environment. This will take a few minutes.

Meenakshi-nodejs-app-env Info

Actions

Upload and deploy

Environment overview

HealthUnknown

Domain-

Environment IDE-h2hih32u8m

Application nameMeenakshi-nodejs-app

Platform

PlatformNode.js 24 running on 64bit Amazon Linux 2023/6.11.3

Running version-

Platform stateSupported

EventsDeploymentsHealth & monitoringLogs NewManaged updatesTags

Events (2) Info

Filter events by text, property or value

Time	Type	Details
July 14, 2026 14:23:14 (UTC+5:30)	INFO	Using elasticbeanstalk-us-east-1-155190455951 as Amazon S3 storage bucket for environment data.

Ignore health checkfalse

Instance replacementfalse

Notifications emailmeenakshimurali2468@gmail.com

Platform software

Lifecyclefalse

Log streamingDisabled

Proxy servernginx

Logs retention7

Rotate logsDisabled

Update levelminor

X-Ray enabledDisabled

Environment properties

Source	Key	Value
No environment properties		
There are no environment properties defined		

environment

Review [Info](#)

service access

Step 1: Configure environment

routing, database, and

Environment information

Environment tier

Web server environment

Application name

Meenakshi-nodejs-app

Environment name

Meenakshi-nodejs-app-env

Application code

Sample application

manage traffic and

Platformarn:aws:elasticbeanstalk:us-east-1::platform/Node.js 24 running
on 64bit Amazon Linux 2023/6.11.3

updates, monitoring, and

Step 2: Configure service access

Service access [Info](#)

Configure the service role and EC2 instance profile that Elastic Beanstalk uses to manage your environment. Choose an EC2 key pair to securely log in to your EC2 instances.

Service role**EC2 instance profile**

Step 2: Configure service access

Edit

Service access

Info

Configure the service role and EC2 instance profile that Elastic Beanstalk uses to manage your environment. Choose an EC2 key pair to securely log in to your EC2 instances.

Service role	EC2 instance profile
arn:aws:iam::155190455951:role/aws-elasticbeanstalk-service-role	aws-elasticbeanstalk-ec2-role

Step 3: Set up networking, database, and tags

Edit

Networking, database, and tags

Info

Configure VPC settings, and subnets for your environment's EC2 instances and load balancer. Set up an Amazon RDS database that's integrated with your environment.

Network	
VPC	Instance subnets
vpc-07aa17451ee13578b	subnet-0420f7f1b9ae91a35,subnet-049dfcee52e35ebf3,subnet-08af9be2d9669e7c9,subnet-08d6b1a7541784534,subnet-0b346af21dfafc9d1,subnet-0dad700c0501a767b

Step 4: Configure instance traffic and scaling

Instance traffic and scaling [Info](#)

Customize the capacity and scaling for your environment's instances. Select security groups to control instance traffic. Configure the software that runs on your environment's instances by setting platform-specific options.

Instances

IMDSv1

Disabled

EC2 Security Groups

sg-0ed42b0631e28c4f3

Capacity

Environment type

Single instance

Fleet composition

On-Demand instance

On-demand base

0

On-demand above base

70

Capacity rebalancing

Disabled

Scaling cooldown

360

Processor type

x86_64

Instance types

c4.large

AMI ID

ami-06a0dff0de1acc86c

Step 5: Configure updates, monitoring, and logging

Step 5: Configure updates, monitoring, and logging

[Edit](#)

Updates, monitoring, and logging [Info](#)

Define when and how Elastic Beanstalk deploys changes to your environment. Manage your application's monitoring and logging settings, instances, and other environment resources.

Monitoring

System

enhanced

Cloudwatch custom metrics - instance

—

Cloudwatch custom metrics - environment

—

Log streaming

Disabled

Retention

7

Lifecycle

false

Updates

Managed updates

Enabled

Deployment batch size

100

Deployment batch size type

Percentage

Command timeout

600

Deployment policy

AllAtOnce

Health threshold

Ok

Ignore health check

false

Instance replacement

false

Notifications email

meenakshimurali2468@gmail.com